



Ultimate Java Backend Interview Blueprint



About ME

Hey, I'm Shubham, 7+ years in tech, 100+ interviews on both sides of the table.

Career path: Infosys → Pubmatic → Amazon → Coursera (currently WFH)

I've cracked multiple high-paying offers across India and internationally using the exact playbook below.

Connect here - [Linkedin](#) , [Instagram](#), [Whatsapp](#)

Java Core (The Foundation)

- JVM Architecture (ClassLoader, Heap, Stack, Metaspace)
 - JDK vs JRE vs JVM
 - Garbage Collection (G1, ZGC, Serial, Parallel — when to use what)
 - Memory Model (Stack vs Heap, Object lifecycle)
 - String Pool & Interning
 - Autoboxing & Unboxing (Integer cache trap)
 - Generics (Wildcards, bounded types, type erasure)
 - Collections Framework (List, Set, Map, Queue — internals)
 - HashMap internals (hashing, collision, resize, load factor)
 - ConcurrentHashMap vs HashMap vs Hashtable
 - Comparable vs Comparator
 - Immutability & final keyword
 - var (Local Type Inference — Java 10+)
 - Records (Java 14+)
 - Sealed Classes (Java 17+)
 - instanceof Pattern Matching
-

OOP & Design Principles

- 4 Pillars (Abstraction, Encapsulation, Inheritance, Polymorphism)
 - SOLID Principles (with real examples)
 - Composition over Inheritance
 - Abstract Class vs Interface (when to use what)
 - Cohesion & Coupling
 - DRY, KISS, YAGNI
 - Method Overloading vs Overriding (runtime vs compile time)
 - Covariant Return Types
 - Static vs Instance methods & variables
-

Multithreading & Concurrency

- Thread Lifecycle (New, Runnable, Blocked, Waiting, Terminated)
 - Runnable vs Callable vs Thread
 - ExecutorService & ThreadPool (Fixed, Cached, Scheduled, Work-Stealing)
 - Future & CompletableFuture (thenApply, thenCompose, allOf, anyOf)
 - synchronized keyword (method vs block)
 - volatile keyword (visibility guarantee)
 - ReentrantLock vs synchronized
 - Deadlock, Livelock, Starvation — detection & prevention
 - Race Conditions & how to avoid
 - Semaphore, CountdownLatch, CyclicBarrier, Phaser
 - ThreadLocal
 - AtomicInteger, AtomicReference (CAS operations)
 - Fork/Join Framework
 - Virtual Threads (Java 21 — Project Loom)
-

Spring Boot & Framework

- Spring IoC Container & Dependency Injection
- Bean Lifecycle (@PostConstruct, @PreDestroy)
- Bean Scopes (Singleton, Prototype, Request, Session)
- @Component vs @Service vs @Repository vs @Controller
- Spring AOP (Aspect, Advice, Pointcut, JoinPoint)
- Spring MVC Request Lifecycle

- @RestController vs @Controller
 - @RequestMapping, @GetMapping, @PostMapping
 - Exception Handling (@ControllerAdvice, @ExceptionHandler)
 - Spring Security (JWT, OAuth2, Filter Chain)
 - Spring Data JPA (Repositories, Query Methods, @Query)
 - Spring Boot Actuator (Health, Metrics, Endpoints)
 - Spring Boot Auto Configuration (how it works internally)
 - Profiles & Environment Config (@Profile, application.yml)
 - Spring Boot Testing (@SpringBootTest, @WebMvcTest, @DataJpaTest)
-

Databases & ORM

- JPA vs Hibernate vs JDBC (when to use what)
 - Entity Lifecycle (Transient, Persistent, Detached, Removed)
 - Lazy vs Eager Loading (N+1 problem & fix)
 - Transactions (@Transactional — propagation, isolation levels)
 - ACID Properties
 - Indexing (B-Tree, Composite, Covering Index)
 - SQL Query Optimization (EXPLAIN, slow query log)
 - Joins (Inner, Left, Right, Full, Self, Cross)
 - Window Functions (ROW_NUMBER, RANK, PARTITION BY)
 - Connection Pooling (HikariCP internals)
 - Database Sharding & Partitioning
 - Read Replicas
 - NoSQL (MongoDB, Redis — when to choose over SQL)
-

System Design & Architecture

- Monolith vs Microservices (trade-offs, when to switch)
- REST API Design Best Practices
- API Versioning Strategies
- gRPC vs REST vs GraphQL
- Event-Driven Architecture (Kafka, RabbitMQ)
- CQRS & Event Sourcing
- Circuit Breaker Pattern (Resilience4j)
- Rate Limiting & Throttling
- Service Discovery (Eureka, Consul)
- API Gateway Pattern
- Saga Pattern (Choreography vs Orchestration)

- Distributed Transactions
 - CAP Theorem
 - Caching Strategies (Cache-aside, Write-through, Write-back)
 - Redis (Data structures, TTL, Pub/Sub, Distributed Lock)
-

Testing

- Unit Testing (JUnit 5 — @Test, @BeforeEach, @AfterEach)
 - Mockito (mock, spy, verify, ArgumentCaptor)
 - Integration Testing (TestContainers)
 - TDD approach
 - Code Coverage (JaCoCo)
 - @MockBean vs @Mock (Spring context vs pure unit)
-

Debugging & Performance

- JProfiler / VisualVM (CPU & Memory profiling)
 - Heap Dump Analysis
 - Thread Dump Analysis (deadlock detection)
 - GC Logs Analysis
 - Slow API debugging (DB query, N+1, connection pool exhaustion)
 - Memory Leak Detection
-

Happy learning ☕😊